

**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em Kotlin

Programação para Aplicativos Móveis

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

tom escuro	#45474B	outer space
acento escuro	#D45113	syracuse red-orange
cor principal	#006674	caribbean current
acento claro	#FFBA08	selective yellow
tom claro	#F5F7F8	anti-flash white

fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono

Orientação a Objetos

classes

- um arquivo pode conter várias classes
- **propriedades** armazenam o estado do objeto
- **funções** definem o comportamento do objeto
- **modificadores de visibilidade** private, internal, protected, public
- **construtores** primários e secundários

exemplos

```
// Java
public class Pessoa {
    private String nome;
    private LocalDate dataNascimento;

    public Pessoa(String nome, LocalDate dataNascimento){
        this.nome = nome;
        this.dataNascimento = dataNascimento;
    }

    public int getIdade(){
        return Period.between(this.dataNascimento,
            LocalDate.now()).getYears();
    }

    public String toString(){
        return String.format("Nome: %s, Data de Nascimento:
        %s, Idade: %d", this.nome, this.dataNascimento,
            this.getIdade());
    }
}
```

Orientação a Objetos

classes

- um arquivo pode conter várias classes
- **propriedades** armazenam o estado do objeto
- **funções** definem o comportamento do objeto
- **modificadores de visibilidade** private, internal, protected, public
- **construtores** primários e secundários

exemplos

```
// Kotlin
class Pessoa(
    var nome: String,
    var dataNascimento: LocalDate
) {

    fun getIdade(): Int {
        return Period.between(dataNascimento,
            LocalDate.now()).years
    }

    override fun toString(): String {
        return "Nome: $nome, Data de Nascimento:
            $dataNascimento, Idade: ${getIdade()}"
    }
}
```

Orientação a Objetos

construtores

- uma classe pode ter construtores primários e secundários
- o construtor **primário** é definido na declaração da classe
- construtores **secundários** são definidos com a palavra-chave constructor
 - construtores secundários devem invocar o primário usando `this()`
- não há suporte a sobrecarga de construtores, mas permite utilizar parâmetros padrão

exemplos

```
// Java
public class Pessoa {
    private String nome;
    private LocalDate dataNascimento;

    public Pessoa(String nome, LocalDate dataNascimento){
        this.nome = nome;
        this.dataNascimento = dataNascimento;
    }

    public Pessoa(String nome) {
        this(nome, LocalDate.now());
    }

    public Pessoa() {
        this("Desconhecido");
    }
}
```

Orientação a Objetos

construtores

- uma classe pode ter construtores primários e secundários
- o construtor **primário** é definido na declaração da classe
- construtores **secundários** são definidos com a palavra-chave **constructor**
 - construtores secundários devem invocar o primário usando `this()`
- não há suporte a sobrecarga de construtores, mas permite utilizar parâmetros padrão

exemplos

```
// Kotlin
class Pessoa(
    var nome: String,
    var dataNascimento: LocalDate
) {

    constructor(nome: String) : this(nome, LocalDate.now())

    constructor() : this("Desconhecido")
}

class Pessoa(
    var nome: String = "Desconhecido",
    var dataNascimento: LocalDate = LocalDate.now()
) { }
```

Orientação a Objetos

getters e setters

- as propriedades são acessíveis diretamente
 - *getters* e *setters* gerados automaticamente
 - propriedades declaradas com `var` têm *getters* e *setters*
 - propriedades declaradas com `val` têm apenas *getters*
- é possível personalizar os *getters* e *setters* se necessário

exemplos

```
// Java
public class Carro {
    private int velocidade = 0;

    public void setVelocidade(int velocidade) {
        if (velocidade < 0) {
            throw new IllegalArgumentException(
                "Velocidade não pode ser negativa");
        }
        this.velocidade = velocidade;
    }

    public int getVelocidade() {
        return (int) velocidade * 0.621371; // km/h para mph
    }
}
```

Orientação a Objetos

getters e setters

- as propriedades são acessíveis diretamente
 - *getters* e *setters* gerados automaticamente
 - propriedades declaradas com `var` têm *getters* e *setters*
 - propriedades declaradas com `val` têm apenas *getters*
- é possível personalizar os *getters* e *setters* se necessário
- `field` é usado para acessar o valor da propriedade

exemplos

```
// Kotlin
class Carro {
    var velocidade: Int = 0
    set(value) { // value como nome do parâmetro
        require(value >= 0) { "Velocidade não pode ser
negativa" }
        field = value
    }
    get() {
        return (field * 0.621371).toInt()
    }
}
```


Orientação a Objetos

getters e setters

- as propriedades são acessíveis diretamente
 - *getters* e *setters* gerados automaticamente
 - propriedades declaradas com `var` têm *getters* e *setters*
 - propriedades declaradas com `val` têm apenas *getters*
- é possível personalizar os *getters* e *setters* se necessário
- `field` é usado para acessar o valor da propriedade
- *setters* podem ser privados

exemplos

```
// Java
public class Carro {
    private int velocidade;
    private void setVelocidade(int velo) { /* impl.*/ }
    public void acelerar(int incremento) { /* impl.*/ }
}

// Kotlin
class Carro(velo: Int) {
    var velocidade: Int = velo.coerceIn(0, VELOCIDADE_MAX)
    private set(value) { /* impl. */ }
    fun acelerar(incremento: Int) { /* impl.*/ }
}
```

Orientação a Objetos

constantes em Java

```
public class Carro {
    private int velocidade;
    private static final int VELOCIDADE_MAX = 200;

    public Carro(int velo){
        this.setVelocidade(velo);
    }

    public void setVelocidade(int velo) {
        velocidade = Math.max(0, Math.min(velo,
            this.VELOCIDADE_MAX));
    }

    public void acelerar(int incremento) {
        if (incremento > 0){
            setVelocidade(velocidade + incremento);
        }
    }
}
```

constantes em Kotlin

```
class Carro(velo: Int) {
    companion object { // equivalente a static em Java
        private const val VELOCIDADE_MAX = 200
    }

    var velocidade: Int = velo.coerceIn(0, VELOCIDADE_MAX)
        set(value) {
            field = value.coerceIn(0, VELOCIDADE_MAX)
        }

    fun acelerar(incremento: Int) {
        if (incremento > 0) {
            velocidade += incremento // chama o setter
        }
    }
}
```

Orientação a Objetos

função *init*

- bloco de inicialização que é executado quando a classe é instanciada
- é possível ter múltiplos blocos `init`
 - executados na ordem em que aparecem

exemplos

```
class Pessoa(val nome: String, val cpf: String) {  
    init {  
        require(validarCpf(cpf)) { "CPF inválido: $cpf" }  
    }  
  
    // função para garantir que o CPF tem 11 dígitos  
    private fun validarCpf(cpf: String): Boolean {  
        val regex = Regex("\\d{11}")  
        return cpf.matches(regex)  
    }  
}
```

Orientação a Objetos

herança

- toda classe herda da classe Any
- por padrão toda classe é final, ou seja, não pode ser estendida
- para permitir herança, a classe deve ser declarada como open
- o mesmo vale para funções
 - para serem sobrescritas, devem ser declaradas como open
 - classes derivadas podem sobrescrever funções com a palavra-chave override
- o construtor da classe base pode ser chamado com super ()

exemplos

```
open class Animal(val nome: String) {  
    open fun fazerSom(): String {  
        return "Som genérico"  
    }  
    fun dormir() {  
        println("$nome está dormindo.")  
    }  
}  
  
class Cachorro(nome: String) : Animal(nome) {  
    override fun fazerSom(): String {  
        return "Au Au"  
    }  
}
```

Orientação a Objetos

classes abstratas

- semelhante a Java
 - declarada com a palavra-chave `abstract`
 - pode conter funções abstratas (sem implementação) e concretas (com implementação)
- não é necessário usar a palavra-chave `open` na classe ou nos métodos abstratos
 - mas é necessário utilizar a palavra-chave `override` para implementar métodos abstratos

exemplos

```
abstract class Animal {  
    abstract fun fazerSom(): String  
    fun dormir() {  
        println("O animal está dormindo.")  
    }  
}
```

Orientação a Objetos

interfaces

- declaradas com a palavra-chave `interface`
- podem conter propriedades e funções, mas não podem ter estado
- funções em interfaces são implicitamente abertos
 - não é necessário usar a palavra-chave `open` na interface ou nos seus métodos
 - mas é necessário utilizar a palavra-chave `override` para implementar métodos
- uma classe pode implementar várias interfaces
- pode fornecer implementações de métodos

exemplos

```
interface Veiculo {  
    fun acelerar()  
    fun frear() {  
        println("Veículo está freando.")  
    }  
}  
  
class Carro : Veiculo {  
    override fun acelerar() {  
        println("Carro está acelerando.")  
    }  
}
```

Orientação a Objetos

implementando várias interfaces

```
abstract class Veiculo(var marca: String, var ano: Int) {  
    override fun toString(): String {  
        return "Marca: $marca, ano: $ano"  
    }  
}  
  
interface Terrestre {  
    fun acelerar()  
}  
  
interface Cargueiro {  
    fun carregar()  
}  
  
class Caminhao(marca: String, ano: Int) : Veiculo(marca, ano), Terrestre, Cargueiro {  
    override fun acelerar() {  
        println("Caminhão está acelerando.")  
    }  
    override fun carregar() {  
        println("Carregando")  
    }  
}
```

Coleções

funções de ordem superior

- **forEach()** itera sobre cada elemento
- **map()** transforma cada elemento aplicando a função passada
- **filter()** retorna um subconjunto da coleção baseado na função filtro
- **reduce()** reduz a coleção a um único valor
- **fold()** similar a reduce, mas passando valor inicial

exemplos

```
numeros.forEach { println(it) }
```

```
val dobrados = numeros.map { it * 2 }
```

```
val pares = numeros.filter { it % 2 == 0 }
```

```
val soma = numeros.reduce { acc, n -> acc + n }
```

```
val soma = numeros.fold(0) { acc, n -> acc + n }
```


Coleções

funções de ordem superior

- **sorted()** ordena os elementos da coleção
- **sortBy()** ordena os elementos da coleção com base na chave definida
- **groupBy()** agrupa os elementos da coleção com base em uma chave

exemplos

```
val ordenados = numeros.sorted()
```

```
val pessoasOrdenadas = pessoas.sortBy { it.idade }
```

```
val agrupados = pessoas.groupBy { it.cidade }
```

Coleções

funções de ordem superior

exemplos

```
class Pessoa(var nome: String, var sobrenome: String)

fun main() {
    val pessoas = listOf(
        Pessoa("Alice", "Silva"),
        Pessoa("Bob", "Santos"),
        Pessoa("Charlie", "Silva")
    )
    // Transformando a lista de pessoas em uma lista de Strings com nomes completos
    val nomesCompleto = pessoas.map { "${it.nome} ${it.sobrenome}" }
    println("Nomes completos: $nomesCompleto")
    // Exemplo de uso de filter
    val pessoasComA = pessoas.filter { it.nome.startsWith("A") }
    // Transformando a lista filtrada em uma lista de Strings com nomes
    println("Pessoas com nome começando com A: ${pessoasComA.map { it.nome }}" )
    // Exemplo de uso de sorted
    val pessoasOrdenadas = pessoas.sortedBy { it.nome }
    // Transformando a lista ordenada em uma lista de Strings com nomes
    println("Pessoas ordenadas por nome: ${pessoasOrdenadas.map { it.nome }}" )
}
```



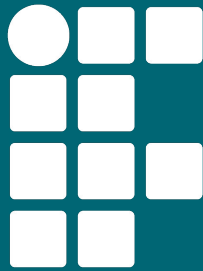
Coleções

funções de ordem superior

exemplos

```
// Exemplo de uso de groupBy
// Retornando um mapa onde a chave é o sobrenome e o valor é uma lista de pessoas com esse sobrenome
// Abaixo foi feito inferência de tipo, então o tipo de pessoasAgrupadas é
// Map<String, List<Pessoa>>
val pessoasAgrupadas = pessoas.groupBy { it.sobrenome }

// Imprimindo as pessoas agrupadas por sobrenome
for ((sobrenome, grupo) in pessoasAgrupadas) {
    println("Sobrenome: $sobrenome")
    for (pessoa in grupo) {
        println(" - ${pessoa.nome}")
    }
}
```



**INSTITUTO
FEDERAL**
Santa Catarina

Orientação a Objetos em Kotlin

Programação para Aplicativos Móveis

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

